

DEFENDING CRITICAL WEB ASSETS

Advanced Perimeter, Scanner Detection,
Bot Management, & API Security.

Disusun Oleh: **Rizki Angga Pratama**
L2 SOC Analyst

HAK CIPTA & DISCLAIMER

Defending Critical Web Assets: Advanced Perimeter & Scanner Detection
Edisi Enterprise (50 Halaman)

Disusun oleh: Rizki Angga Pratama, S. Kom, C. Pro

Dokumen ini memfokuskan diri pada seni bertahan di Garis Depan (Edge). Buku ini adalah panduan teknis bagi analis SOC untuk membaca log WAF, membedakan pengguna sah dari *Botnet*, dan merancang aturan pemblokiran IP yang dinamis.

> **Catatan Operasional:**
> Buku ini menggunakan referensi log dari *Web Application Firewall (WAF)* modern (seperti Cloudflare, F5, AWS WAF, dan NGINX). Analitik SIEM (KQL) yang disajikan ditujukan untuk platform Elastic Security atau Kibana, namun logikanya dapat diterjemahkan ke Splunk SPL atau Microsoft Sentinel KQL.

Filosofi Desain:
Dirancang menggunakan estetika Cyber Matrix (Emerald Green & Sky Blue). Warna-warna ini mewakili elemen protokol Web (HTTP 200 OK), perisai WAF (Shields), dan validasi kriptografi di lapisan transportasi.

© 2026 Hak cipta dilindungi undang-undang. Dokumen referensi pelatihan SOC.

KATA PENGANTAR

Aplikasi web dan API publik Anda adalah etalase digital perusahaan. Mereka hidup di jalanan internet yang paling bising dan paling kejam. Dalam satu menit peladen web Anda daring (online), ia akan dipindai, diuji, dan diserang oleh ribuan skrip otomatis dari seluruh dunia.

Bertahan di perimeter ini tidak lagi semudah memasang Firewall port 80/443. Peretas masa kini tidak menyerang jaringan (Lapis 3/4); mereka menyerang logika aplikasi (Lapis 7). Mereka membaur dengan lalu lintas normal menggunakan *Residential Proxies*, mereka mengekstrak data perlahan (Low and Slow), dan mereka memanipulasi parameter API yang tidak pernah Anda pikirkan.

Dalam e-book 50 halaman ini, kita akan mengubah posisi Anda dari sekadar pengamat pasif menjadi arsitek pertahanan proaktif. Kita akan membedah anatomi alat peretas seperti Nuclei dan Burp Suite, merancang taktik Rate Limiting tingkat lanjut, mendeteksi Credential Stuffing, dan menggunakan Threat Intelligence untuk membangun tembok api dinamis.

Fokus kita adalah mengeliminasi kebisingan (*Noise*). Jika kita dapat menghentikan 95% pemindai otomatis (Bot Scanners) di tepi jaringan (Edge), tim SOC Anda dapat memfokuskan energi mereka pada 5% serangan APT (*Advanced Persistent Threat*) yang dikendalikan oleh manusia nyata.

— Rizki Angga Pratama

Author

DAFTAR ISI (1/2)

BAGIAN I: Anatomi Serangan Web & Perimeter

Bab 1.1 Lanskap Ancaman Web Modern Hlm. 7

Bab 1.2 OWASP Top 10 & Evolusinya Hlm. 8

Bab 1.3 Konsep Edge Computing & Reverse Proxy Hlm. 9

Bab 1.4 Web Application Firewall (WAF) 101 Hlm. 10

Bab 1.5 Positive vs Negative Security Models Hlm. 11

Bab 1.6 Proteksi L3/L4 vs L7 di Perimeter Hlm. 12

BAGIAN II: Mendeteksi Pemindai (Reconnaissance)

Bab 2.1 Anatomi Web Scanners (Nuclei, Nikto) Hlm. 14

Bab 2.2 Analisis Anomali User-Agent Hlm. 15

Bab 2.3 Pola HTTP Error (404/403 Spikes) Hlm. 16

Bab 2.4 KQL: Deteksi Direktori Brute Force Hlm. 17

Bab 2.5 Active Defense: Penggunaan Honeytokens Hlm. 18

Bab 2.6 Mendeteksi Path Traversal & LFI Hlm. 19

Bab 2.7 KQL: Deteksi Payload SQLi di URI/Body Hlm. 20

Bab 2.8 Mengelola False Positives dari WAF Hlm. 21

Bab 2.9 Dynamic IP Banning (Fail2Ban Logic) Hlm. 22

DAFTAR ISI (2/2)

BAGIAN III: DDoS Mitigation & Bot Management

Bab 3.1 Anatomi Serangan L7 (Application DDoS) Hlm. 24

Bab 3.2 Asymmetric Attacks (Slowloris/Slow POST) Hlm. 25

Bab 3.3 Strategi Rate Limiting Lanjutan Hlm. 26

Bab 3.4 KQL: Mendeteksi Serangan Volumetrik L7 Hlm. 27

Bab 3.5 Tantangan Bot Management Modern Hlm. 28

Bab 3.6 JS Challenges, Turnstile, & CAPTCHA Hlm. 29

Bab 3.7 Credential Stuffing (Account Takeover) Hlm. 30

Bab 3.8 KQL: Deteksi Credential Stuffing Hlm. 31

Bab 3.9 Analisis TLS Fingerprinting (JA3/JA3S) Hlm. 32

BAGIAN IV: API Security & Advanced Vectors

Bab 4.1 Mengapa API Berbeda dari Web Konvensional? Hlm. 34

Bab 4.2 BOLA (Broken Object Level Authorization) Hlm. 35

Bab 4.3 KQL: Mendeteksi Anomali Iterasi API Hlm. 36

Bab 4.4 API Gateway & Schema Validation (OAS) Hlm. 37

Bab 4.5 Eksploitasi & Manipulasi JWT Hlm. 38

Bab 4.6 Server-Side Request Forgery (SSRF) Hlm. 39

Bab 4.7 KQL: Deteksi SSRF pada Log WAF Hlm. 40

Bab 4.8 Keamanan GraphQL (Introspection Abuse) Hlm. 41

BAGIAN V: SOC Operations untuk Web Assets

Bab 5.1 Ingesting Web Logs (NGINX/CDN) ke SIEM Hlm. 43

Bab 5.2 Normalisasi ECS (Elastic Common Schema) Hlm. 44

Bab 5.3 Merancang Dashboard Visibilitas Perimeter Hlm. 45

Bab 5.4 Otomatisasi SOAR: WAF Blocklist Update Hlm. 46

Bab 5.5 External Dynamic Lists (EDL) & Threat Intel Hlm. 47

Glosarium Web Security (A-N) Hlm. 48



BAGIAN I

Anatomi Serangan Web
& Pertahanan Perimeter

BAB 1: FONDASI PERIMETER

1.1 Lanskap Ancaman Web Modern

Internet publik adalah zona perang tanpa hukum. Dalam beberapa menit setelah Anda meluncurkan aplikasi web baru ke alamat IP publik (AWS EC2 atau DigitalOcean), log akses peladen Anda akan mulai dipenuhi dengan permintaan aneh. Permintaan ini bukan dari manusia, melainkan dari *Botnets*, mesin pemindai (*scanners*) otomatis, dan alat eksploitasi skrip milik penyerang oportunis.

Ancaman web modern terbagi menjadi tiga kategori utama:

- **Volumetric Attacks:** Serangan *Distributed Denial of Service* (DDoS) yang bertujuan menghabiskan *bandwidth* jaringan atau sumber daya CPU peladen Anda hingga aplikasi *crash* (padam).
- **Logic Attacks (Eksploitasi L7):** Percobaan menyuntikkan kode berbahaya (SQLi, XSS, RCE) melalui formulir isian (*input*), *header HTTP*, atau parameter URI untuk mencuri data dari basis data di belakang aplikasi.
- **Automated Abuse (Bot):** Penyalahgunaan fitur yang secara teknis "legal". Contoh: Memborong tiket konser dalam 1 detik (*Scalping*), menguji jutaan kata sandi curian di halaman *login* (*Credential Stuffing*), atau menyedot konten harga produk (*Web Scraping*).

1.2 OWASP Top 10 & Evolusinya

Bibel (Kitab Suci) bagi para insinyur keamanan aplikasi adalah **OWASP Top 10** (*Open Worldwide Application Security Project*). Ini adalah dokumen kesadaran yang mewakili 10 kerentanan aplikasi web paling kritis.

Selama dekade terakhir, daftar ini telah berevolusi, mencerminkan pergeseran teknologi dari aplikasi *Monolithic* (satu peladen raksasa) menuju arsitektur *Microservices* dan *API-driven* (berbasis API).

Misalnya, **Injection** (seperti SQL Injection) yang dulu merajai nomor 1, kini mulai tergeser oleh **Broken Access Control** (Kegagalan Kontrol Akses). Penyerang modern menyadari bahwa meretas *Database* menggunakan SQLi (yang dihalau kuat oleh WAF dan ORM modern) terlalu sulit. Lebih mudah memanipulasi *API Endpoint* (misalnya mengubah parameter URL `/api/user/101` menjadi `/api/user/102`) untuk membaca profil orang lain tanpa izin. Celah logika akses semacam ini seringkali lolos dari deteksi WAF tradisional.

1.3 Edge Computing & Reverse Proxy

Secara arsitektur, Anda tidak boleh membiarkan pengguna internet terhubung secara langsung (*Direct TCP Connection*) ke peladen aplikasi asli (*Origin Server*) Anda. Sebagai gantinya, arsitektur modern menggunakan proksi terbalik (**Reverse Proxy**) di tepi jaringan (**Edge**).



Jaringan *Edge* (seperti Cloudflare, Fastly, atau AWS CloudFront) menyerap miliaran lalu lintas, melakukan dekripsi sertifikat SSL/TLS (HTTPS), memeriksa setiap *byte* paket menggunakan mesin aturan (*Rules Engine*), lalu merelai paket yang bersih (*Clean Traffic*) ke peladen *Origin* Anda.

Bagi peretas, alamat IP peladen *Origin* Anda harus tetap tersembunyi. Jika peretas mengetahui IP asli Anda, mereka dapat membidik peladen Anda langsung, memintas (*bypass*) perlindungan *Edge WAF* sepenuhnya.

1.4 Web Application Firewall (WAF) 101

Web Application Firewall (WAF) adalah perisai utama dalam pertahanan Lapis 7. Berbeda dengan *Network Firewall* tradisional yang hanya melihat IP dan *Port* (Lapis 3/4), WAF mengerti "bahasa" aplikasi web (HTTP/HTTPS).

WAF membedah dan menganalisis setiap komponen dari permintaan HTTP (*HTTP Request*):

- **URI Path & Query String:** Mencari injeksi (misal: `?id=1' OR '1'='1`).
- **HTTP Headers:** Mencari agen pengguna (*User-Agent*) jahat, atau injeksi di *header* Referer.
- **POST Body (Payload):** Menganalisis data formulir, dokumen JSON, atau XML untuk mencari struktur kode berbahaya, *malware* yang diunggah (RCE), atau pelanggaran validasi skema.

Saat paket HTTP terdeteksi cocok dengan tanda tangan (*Signature*) serangan dalam basis datanya, WAF akan memberikan tindakan, biasanya berupa blokir (*Block*) dengan kode status **HTTP 403 Forbidden**.

1.5 Positive vs Negative Security Models

Konfigurasi WAF berputar di antara dua filosofi kontrol yang berbeda:

Negative Security Model (Blacklisting):
WAF mengizinkan *semua* lalu lintas secara *default*, KECUALI yang cocok dengan daftar hitam (*Blacklist/Signatures*).
Kelebihan: Mudah diimplementasikan, jarang memblokir pengguna sah (False Positive rendah).
Kelemahan: Buta terhadap serangan baru (*Zero-Day Exploit*) yang belum ada di daftar hitam.

Positive Security Model (Whitelisting):
WAF **memblokir semua** lalu lintas secara *default*, KECUALI yang benar-benar mematuhi aturan spesifik.
Kelebihan: Keamanan absolut. Jika API Anda hanya menerima data "Angka" (*Integer*), WAF akan otomatis memblokir injeksi "Huruf/Skrip", meniadakan serangan *Zero-Day*.
Kelemahan: Sangat sulit dipelihara (*High Maintenance*). Pengembang harus mendefinisikan skema secara presisi; jika tidak, aplikasi akan rusak dan pengguna sah terblokir.

Kebanyakan WAF modern tingkat perusahaan (*Enterprise*) menggunakan kombinasi keduanya (*Hybrid*).

1.6 Proteksi L3/L4 vs L7 di Perimeter

Seorang analis SOC harus bisa membedakan di lapisan OSI (*Open Systems Interconnection*) mana sebuah serangan beroperasi, karena mitigasinya membutuhkan alat yang berbeda.

FITUR SERANGAN	NETWORK LAYER (L3/L4)	APPLICATION LAYER (L7)
Contoh Serangan	SYN Flood, UDP Reflection (NTP/DNS Amplification).	HTTP GET Flood, SQL Injection, API Abuse.
Tujuan	Menyumbat pipa *Bandwidth* jaringan (Saturasi Pipa).	Menghabiskan CPU/RAM Peladen Web & Database.
Alat Mitigasi	BGP Anycast, Router *Null Routing*, *Scrubbing Centers*.	WAF, CAPTCHA, *Rate Limiting* IP, Bot Management.
Visibilitas SOC	Log *NetFlow* / *Packet Capture* (PCAP).	Log WAF / Log Akses NGINX (Format JSON).

Serangan L3/L4 bersifat **Volumetrik Raksasa** (Terabit per detik), biasanya diurus secara otomatis oleh *router* CDN. Tim SOC internal jarang terlibat. Sebaliknya, serangan L7 ukurannya kecil namun sangat merusak logika aplikasi. Di sinilah tim SOC (Deteksi & Respons) menghabiskan 90% waktu operasional mereka.



BAGIAN II

Mendeteksi Pemindai (Scanners)
& Menghentikan Reconnaissance

BAB 2: DETEKSI PEMINDAI WEB

2.1 Anatomi Web Scanners (Nuclei, Nikto)

Sebelum peretas mengeksploitasi, mereka harus menemukan celah. Mereka tidak melakukannya secara manual (mengklik situs Anda). Mereka menggunakan alat pemindai otomatis (*Vulnerability Scanners*).

Alat legendaris seperti **Nikto** atau **Burp Suite (Active Scan)**, dan alat modern favorit para *Bug Bounty Hunters* seperti **Nuclei**, bekerja dengan cara membombardir ribuan *Endpoint URL* peladen Anda dengan beragam templat *Payload* (isian berbahaya).

Pemindai ini akan secara brutal mencari berkas konfigurasi lingkungan (`/.env`), *folder* riwayat Git (`/.git/config`), atau mencoba injeksi CVE spesifik (seperti celah *Log4j*). Karena mereka beroperasi secara berurutan dan cepat (ratusan *Request* per detik), alat-alat ini meninggalkan **jejak akustik (Acoustic Signature)** yang sangat bising di dalam Log Akses Web Anda.

2.2 Analisis Anomali User-Agent

Celah paling sederhana (dan sering kali ceroboh) dari skrip pemindai yang dikonfigurasi dengan buruk adalah tajuk **User-Agent (UA)**. *User-Agent* adalah identitas peramban (*browser*) yang dilaporkan klien ke peladen (misal: `Mozilla/5.0 (Windows NT 10.0...) Chrome/114.0...`).

Banyak pemindai sumber terbuka (*Open Source*) dan *bot* Python menggunakan *User-Agent* bawaan (*default*) pembuatnya, yang membuatnya sangat mudah diblokir oleh WAF.

> **Contoh User-Agent Berbahaya/Anomali (Blacklist):**
> - `python-requests/2.28.1` (Skrip otomatis Python)
> - `curl/7.68.0` (Pengujian Terminal)
> - `Nuclei - Open-source project` (Pemindai Nuclei)
> - `Nikto/2.1.6` (Pemindai Nikto)
> - `Mozilla/5.0 (compatible; Nmap Scripting Engine)` (Nmap)

SOC Analyst harus membuat dasbor SIEM yang menampilkan agregasi 10 *User-Agent* teratas. Jika ada lonjakan trafik HTTP masuk tanpa *User-Agent* (kosong / `-`) atau menggunakan UA aneh di atas, blokir IP sumbernya secara instan.

2.3 Pola HTTP Error (404/403 Spikes)

Namun, peretas yang cerdas akan memalsukan (*Spoof*) *User-Agent* mereka agar terlihat seperti *Google Chrome* atau *iPhone*. Jika mereka menyamar, bagaimana kita mendeteksinya?

Jawabannya ada pada **HTTP Status Codes**. Manusia normal menavigasi situs dengan mengklik tautan (*link*). Mereka jarang tersesat. Oleh karena itu, manusia sebagian besar menghasilkan kode **200 OK**.

Pemindai menebak (*Brute Force*) nama direktori secara membabi buta (misal: menebak `/admin`, `/backup.zip`, `/phpinfo.php`). Mayoritas tebakan ini akan salah, sehingga peladen web Anda akan merespons dengan kode **404 Not Found**. Jika pemindai mengenai area terlarang, peladen menjawab **403 Forbidden**.

Tanda tangan absolut (*Ultimate Signature*) dari pemindai/reconnaissance otomatis adalah: **Rasio Error (404) terhadap Sukses (200) yang ekstrem dari satu alamat IP dalam waktu singkat.**

2.4 KQL: Deteksi Direktori Brute Force

Kita dapat menerjemahkan anomali kode status HTTP 404 (*Not Found Spikes*) ke dalam analitik SIEM (Kibana / Elastic) untuk melahirkan deteksi peringatan *High Severity*.

```
/* SIEM KQL: Deteksi Web Directory Brute Force / DirBuster */ event.dataset: "nginx.access" AND
http.response.status_code: 404 /* Agregasi: IP menghasilkan lebih dari 50x error 404 dalam 1 menit */ |
stats count() as error_count by source.ip | where error_count > 50 | timeframe 1m
```

Peringatan False Positive: Jika situs web Anda baru saja melakukan pembaruan (*Redesign*) dan banyak gambar/CSS lama yang terhapus, pengguna normal mungkin akan memicu ratusan pesan 404 saat memuat ulang halaman (*broken links*). Untuk menanganinya (*Tuning*), Anda harus mengabaikan ekstensi aset statis dalam aturan KQL Anda: `AND NOT url.path: (*.jpg OR *.css OR *.png)`.

2.5 Active Defense: Penggunaan Honeytokens

Strategi tercanggih dalam mendeteksi pemindai bukan sekadar memblokir, melainkan menggunakan **Deception (Penipuan)** atau *Active Defense*. Kita menanam **Honeytokens** (Jebakan Madu).

Tim *DevSecOps* sengaja membuat tautan (URL) rahasia atau *file* palsu di *root* direktori web. Contohnya: `/config_db_backup_2026.bak` atau `/admin-legacy/login.php`. Tautan ini **tidak pernah dipublikasikan** dan tidak dapat dijangkau oleh pengguna sah, apalagi di-*crawl* oleh GoogleBot (karena disembunyikan via `robots.txt` dengan status *Disallow*).

Jika ada entitas (IP) mana pun di alam semesta ini yang mencoba mengakses *URL Honeytoken* tersebut, tidak perlu dilakukan agregasi rasio, dan tidak ada investigasi lebih lanjut. Niat IP tersebut 100% adalah peretasan (*Malicious*). WAF/SIEM dikonfigurasi untuk menangkap *hit* pada URI tersebut dan langsung memasukkan IP penyerang ke dalam **Global Blacklist** secara permanen.

2.6 Mendeteksi Path Traversal & LFI

Selain meraba-raba (*recon*) struktur, pemindai akan mencoba membaca *file* sistem bawaan Linux untuk menguji celah **Local File Inclusion (LFI)** atau **Path Traversal**.

Dalam serangan ini, penyerang memanipulasi parameter URL (seperti fitur unduh gambar) untuk keluar dari keranjang *folder* web (*sandbox*), bergerak mundur (ke atas *folder* akar), dan menyedot *file* konfigurasi sensitif OS, paling sering adalah fail sandi: `/etc/passwd` (Linux) atau `boot.ini` (Windows).

> **Payload LFI Klasik:**
> `GET /download.php?file=../../../../../../etc/passwd`
> `GET /image?path=..%2f..%2f..%2fetc%2fpasswd` (URL Encoded)

WAF mendeteksi serangan ini dengan mencari pola Regular Expression (RegEx) `\.\./` (titik-titik-garis miring) secara berulang. WAF modern juga melakukan normalisasi ganda (*Double URL Decoding*) untuk membongkar trik penyamaran *encoding* sebelum mencocokkannya dengan aturan blokir.

2.7 KQL: Deteksi Payload SQLi di URI

SQL Injection (SQLi) tetap menjadi prioritas tinggi. WAF memblokir ribuan serangan SQLi setiap jam. Sebagai analis SOC, Anda perlu merangkum dan mencari tahu apakah serangan tersebut **berhasil (Success)** lolos dari WAF.

Jika WAF berada dalam mode "Pemantauan Saja" (*Log Only / Transparent Mode*), Anda harus mengandalkan SIEM untuk memicu peringatan.

```
/* SIEM KQL: Deteksi Keyword SQL Injection pada URL */ event.dataset: "apache.access" AND url.query: ( "*UNION*SELECT*" OR "*WAITFOR*DELAY*" OR "*1=1*" OR "*OR*'a'='a*" ) /* Kritis: Kita mencari HTTP 200 (Eksploitasi Sukses Tereksekusi!) */ AND http.response.status_code: 200
```

Kata kunci `WAITFOR DELAY` adalah vektor *Time-Based Blind SQLi*. Kueri di atas adalah contoh mengapa *SIEM Correlation* penting. Menangkap SQLi di Log Akses adalah bukti bahwa lapisan WAF Anda gagal memblokir permintaan berbahaya tersebut.

2.8 Mengelola False Positives WAF

Keluhan terbesar dari *Developer* (Pengembang) terhadap tim *Security* adalah **False Positive (Positif Palsu)**: WAF memblokir pengguna asli atau transaksi bisnis yang sah karena salah duga sebagai serangan.

Contoh klasik: Aplikasi CMS (seperti WordPress) mengharuskan penulis (*Blogger*) mengunggah artikel yang berisi potongan kode HTML atau *Javascript*. Saat artikel tersebut dikirim via formulir (HTTP POST), WAF melihat *script* HTML, mengiranya sebagai serangan *Cross-Site Scripting (XSS)*, dan memblokir publikasi artikel tersebut.

Untuk menyelesaikannya, analis SOC harus melakukan penyesuaian aturan (**WAF Tuning & Exceptions**). Alih-alih mematikan seluruh perlindungan XSS, analis membuat aturan *Bypass* spesifik: *"Matikan pengecekan XSS HANYA untuk alamat IP milik Kantor Pusat, DAN HANYA saat mereka melakukan HTTP POST ke URI `/wp-admin/post.php`. Tetap lindungi seluruh pengguna lain secara global."*

2.9 Dynamic IP Banning (Fail2Ban)

Pertahanan statis tidak lagi memadai. Jika sebuah alamat IP pemindai menabrak (*hit*) aturan WAF Anda sebanyak 50 kali dalam satu menit dan mendapatkan respons "403 Forbidden", mengapa Anda harus membiarkan WAF terus memproses permintaan ke-51 dan seterusnya? (Memproses *rules* membutuhkan daya komputasi/CPU).

Arsitektur *Enterprise* mengadopsi **Dynamic IP Banning** (terinspirasi dari perangkat lunak *Fail2Ban*). Sistem SOAR terhubung ke log WAF secara *real-time*.

Jika *IP 192.168.10.10* memicu lebih dari 30 *WAF Alert* (SQLi/XSS/LFI) dalam durasi 5 menit, SOAR akan memicu *API Call* ke lapisan L3 (*Edge Router / Cloudflare IP Firewall*) untuk menjatuhkan (*Drop*) paket jaringan dari IP tersebut secara absolut di level TCP, sebelum paket HTTP terbentuk.

Blokir dinamis ini (*Jail*) diberikan *Time-To-Live (TTL)* selama 4 jam. Setelah 4 jam, IP tersebut dibebaskan (*Unbanned*). TTL penting karena alamat IP bisa jadi adalah IP Publik NAT bersama (seperti jaringan penyedia seluler), dan memblokirnya permanen dapat melumpuhkan akses pelanggan sah (*Collateral Damage*).



BAGIAN III

DDoS Mitigation L7, CAPTCHA,
& Bot Management

BAB 3: BOTNETS & DDOS

3.1 Anatomi Serangan L7 (Application DDoS)

Jika serangan DDoS L3 (Network) menargetkan ukuran pipa (misal: mengirimkan *junk traffic* 100 Gbps agar *router* ISP hancur), **DDoS Application Layer (L7)** menargetkan otak peladen web (CPU/RAM/Database). Serangan L7 sangat menakutkan karena mereka **berukuran kecil dan terlihat sangat sah (HTTP 200 OK)**.

Contohnya adalah **HTTP GET Flood**. 10.000 komputer *Zombie/Botnet* yang diretas dari seluruh dunia mengirimkan permintaan HTTP GET serentak ke fitur pencarian web (*Search Bar*) aplikasi Anda (misal: `/search?q=produk`). Karena fitur pencarian mengharuskan peladen Web membangun kueri ke peladen *Database* untuk setiap *request*, hal ini sangat membebani komputasi (CPU).

Meskipun secara jaringan (*Bandwidth*) trafiknya hanya 50 Mbps (sangat kecil), namun 50.000 permintaan pencarian basis data per detik akan membuat *Database Server* berasap, *CPU Usage 100%*, dan aplikasi menjadi *Down* (Tidak dapat diakses).

3.2 Asymmetric Attacks (Slowloris/Slow POST)

Variasi DDoS L7 yang lebih cerdik dan senyap disebut **Low and Slow Asymmetric Attacks**. Alat populernya adalah *Slowloris* atau *R-U-Dead-Yet (RUDY)*.

Dalam serangan **Slowloris**, penyerang memanfaatkan arsitektur keterbatasan (*Connection Pool*) dari peladen *web* lawas (seperti *Apache Thread-Based*). Peladen hanya dapat melayani maksimal 500 koneksi pengguna secara bersamaan.

Penyerang membuka 500 koneksi HTTP secara berbarengan, namun mereka tidak pernah menyelesaikan permintaannya. Mereka mengirimkan data sangat, sangat, perlahan (misal: 1 *Byte* setiap 10 detik). Peladen harus membiarkan (menahan) 500 koneksi itu tetap terbuka untuk menunggu kelanjutan paket.

Akibatnya, slot koneksi (*Sockets*) peladen habis. Saat pengguna asli (pelanggan sah) mencoba masuk, peladen tidak memiliki ruang (*threads*) tersisa untuk melayaninya, sehingga situs seolah-olah mati. Ini dicapai hanya oleh 1 laptop peretas tanpa perlu *bandwidth* besar.

3.3 Strategi Rate Limiting Lanjutan

Obat mujarab untuk HTTP *Floods* dan upaya *brute-force* otomatis adalah pembatasan laju, atau **Rate Limiting**.

Aturan *Rate Limiting* Dasar membatasi berdasarkan IP (Misal: *1 IP maksimal 100 request/menit*). Namun, peretas memintasnya dengan memutar IP menggunakan ribuan *Residential Proxies* (setiap *request* menggunakan IP negara berbeda).

Oleh karena itu, *SOC Engineer* merancang **Advanced Rate Limiting** yang membatasi berdasarkan properti gabungan (*Composite Keys*) HTTP, bukan sekadar IP:

> **Contoh Skenario Advanced Rate Limit:**
> 1. **Berdasarkan URI Sentris:** Batasi akses ke *endpoint* berat (seperti `/api/v1/generate-pdf`) maksimal 5 *request* per menit, tanpa peduli IP apa pun.
> 2. **Berdasarkan ASN/Negara:** Jika 80% pelanggan Anda berada di Indonesia, terapkan pembatasan laju yang sangat agresif (misal 10 *req/min*) untuk blok ASN yang berasal dari Tiongkok atau Rusia.
> 3. **Berdasarkan Header Autentikasi (JWT):** Batasi laju berdasarkan `UserID` yang diekstrak dari Token API, menetralkan rotasi alamat IP peretas.

3.4 KQL: Mendeteksi Serangan Volumetrik L7

Saat terjadi insiden aplikasi *Down* (Mati), SOC *Analyst* harus segera melakukan investigasi di SIEM untuk menentukan: Apakah ini kesalahan pembaruan *Developer* (Bug Kode), atau kita sedang diserang DDoS L7?

```
/* SIEM KQL: Mendeteksi L7 HTTP Flood Spikes (Volumetrik) */ event.dataset: "nginx.access" /* Hitung jumlah Request HTTP per URI (Fitur) yang dihajar */ | stats count() as hit_count by url.path /* Agregasi visual: Tampilkan URI yang diakses > 5000 kali dalam 2 menit */ | where hit_count > 5000 | timeframe 2m | sort hit_count desc
```

Jika kueri di atas tiba-tiba memunculkan lonjakan puluhan ribu *hit* pada `url.path: /login`, Anda memiliki bukti kuat serangan *Botnet*. Tindakan operasional seketika (*Incident Response*) adalah memasang mode *"Under Attack"* pada *dashboard* Cloudflare/CDN untuk URI `/login` tersebut.

3.5 Tantangan Bot Management Modern

Seiring evolusi WAF, *Botnets* juga berevolusi. Bot generik (seperti CURL atau skrip Python dasar) sangat mudah diblokir. Namun, era komputasi melahirkan **Headless Browsers** (seperti *Puppeteer* atau *Selenium*).

Headless Browsers adalah mesin *Google Chrome* asli yang dikendalikan oleh kode (*script*), bukan manusia. Karena ini adalah *browser* asli, alat ini mampu memproses *Javascript* dan me-*render* halaman, sehingga WAF tradisional sering kali tertipu dan menganggapnya manusia.

Untuk melawannya, vendor *Edge* (seperti Akamai Bot Manager atau Cloudflare Bot Management) menginjeksikan mesin **Fingerprinting/Telemetry Javascript** secara rahasia ke dalam halaman HTML. Mesin ini mengumpulkan sinyal biometrik digital: Apakah pergerakan *Mouse* natural? Apakah pengetikan *Keyboard* (*Keystrokes*) memiliki jeda manusia (milidetik yang tidak konstan)? Apakah resolusi layar masuk akal? Berdasarkan ratusan parameter ini, sistem memberikan Skor *Bot (Bot Score: 1-100)*.

3.6 JS Challenges, Turnstile, & CAPTCHA

Bagaimana respons sistem jika permintaan (*Request*) menerima *Bot Score* yang rendah (mencurigakan)? Sistem perlindungan akan melempar **Tantangan (Challenge)**.

- **JS Challenge (Tantangan Tak Kasat Mata):** WAF tidak langsung memberikan halaman web. Ia mengirimkan potongan kode Javascript rumit (operasi matematika) yang harus diselesaikan (*render/compute*) oleh mesin klien dalam beberapa detik. *Bot* skrip murah akan gagal di sini. Pengguna asli tidak akan sadar ini terjadi.
- **Managed Challenge / Turnstile:** Teknologi anti-bot nirgesekan (*Frictionless*). Hanya berupa kotak centang (*Checkbox*) "Saya bukan robot", yang menganalisis telemetri peramban di latar belakang tanpa puzzle.
- **Interactive CAPTCHA:** Pilihan terakhir jika lalu lintas sangat meragukan. Pengguna dipaksa memilih gambar (seperti "Pilih semua gambar Lampu Lalu Lintas"). Sangat menjengkelkan pengguna, sehingga digunakan sebagai batas akhir (*Last Resort*) untuk lalu lintas anomali di bawah skor 30.

3.7 Credential Stuffing (Account Takeover)

Vektor serangan *Bot* yang paling diuntungkan secara finansial adalah **Credential Stuffing (ATO - Account Takeover)**.

Peretas membeli 10 Juta basis data (*Username* dan *Password*) hasil kebocoran situs *Lain* di pasar gelap. Karena manusia sering mengulang (*Reuse*) *password* yang sama di berbagai aplikasi, peretas memprogram *Botnet* raksasa untuk menguji (mencoba masuk) ke portal aplikasi E-Commerce atau *Banking* Anda menggunakan 10 juta data curian tersebut.

Karena mereka memutar (*Rotate*) Alamat IP menggunakan proksi yang menyebar di seluruh dunia, dan setiap IP hanya mencoba *login* 1 kali setiap jam, aturan *Rate Limiting* tradisional **TIDAK AKAN MENDETEKSI MEREKA**. Serangan ini sangat sunyi (*Low and Slow*).

Jika berhasil (sandi ternyata sama), *Bot* akan menyedot saldo akun korban atau poin loyalitas (*Rewards*).

3.8 KQL: Deteksi Credential Stuffing

Menangkap *Credential Stuffing* yang didistribusikan (*Distributed*) memerlukan perubahan cara pandang *Correlation Rule* di SIEM. Kita tidak memantau IP-nya (karena IP selalu berubah), kita memantau **End Point Login-nya** secara keseluruhan.

```
/* SIEM KQL: Deteksi Distributed Credential Stuffing */ event.dataset: "waf.access" AND url.path: "/api/v1/auth/login" AND http.response.status_code: 401 /* Unauthorized / Salah Sandi */ /* Agregasi: Jika Total GLOBAL gagal login > 5.000x, namun berasal dari lebih dari 2.000 IP unik. Ini mengonfirmasi serangan Botnet Global. */ | timeframe 15m | stats count() as TotalGagal, count(distinct source.ip) as IP_Unik | where TotalGagal > 5000 AND IP_Unik > 2000
```

Solusi arsitektural untuk ATO adalah memaksakan **Multi-Factor Authentication (MFA)** pada semua pengguna. Meskipun *Botnet* menebak *password* dengan benar, mereka tidak dapat melangkah masuk tanpa token OTP dari HP pengguna asli.

3.9 Analisis TLS Fingerprinting (JA3/JA3S)

Salah satu trik penyamaran *Bot* adalah menyamarkan *User-Agent* (Bab 2.2). *Bot Python* dapat mengklaim dirinya sebagai `Chrome/114`. Bagaimana cara kita mengetahui ia berbohong?

Arsitek SOC menggunakan teknik **JA3 TLS Fingerprinting**. Saat aplikasi berkomunikasi melalui HTTPS (SSL/TLS), terjadi proses jabat tangan (*TLS Handshake / Client Hello*). Algoritma matematika dan *Cipher Suites* yang ditawarkan oleh *Chrome* asli SANGAT BERBEDA dengan perpustakaan (*Library*) `urllib` bawaan *Python* atau *Golang*.

JA3 mengumpulkan metode kriptografi (*Cipher, Extensions, Elliptic Curves*) dari koneksi tersebut, dan mengubahnya menjadi *Hash MD5* yang unik. (Misal: `Hash JA3 Chrome: 771,4865...`, `Hash JA3 Python: 443,491...`).

Bahkan jika *script Bot* itu memalsukan *User-Agent* miliknya sebagai *Google Chrome*, nilai *Hash JA3* yang dikirimkan di level jaringan (karena *Library* Python) **tidak bisa dipalsukan**. WAF *Edge* modern mencocokkan anomali ketidaksesuaian (*Mismatch*) ini dan memblokir paket TLS di lapisan 4 secara absolut.



BAGIAN IV

API Security &
Advanced Attack Vectors

BAB 4: KEAMANAN API

4.1 Mengapa API Berbeda dari Web Biasa?

Modernisasi arsitektur (Web 2.0/3.0 dan Aplikasi *Mobile*) telah menyingkirkan halaman HTML statis dan menggantinya dengan **Application Programming Interfaces (API)**, khususnya RESTful dan GraphQL.

API mengalirkan data murni (biasanya dalam format struktur data JSON) secara langsung. Hal ini menciptakan dilema keamanan yang sangat berbeda:

- **Hilangnya Perimeter Formulir:** Dulu, peretas harus melalui kotak isian HTML. Sekarang, mereka mengirim perintah mesin-ke-mesin langsung ke titik akhir (*Endpoints*).
- **Logika Data Mentah:** API terlalu percaya diri (*Over-fetching*). Seringkali *Frontend* (Aplikasi HP) hanya meminta nama *User*, namun API *Backend* merespons dan mengirimkan JSON raksasa berisi (Nama, Alamat, KTP, dan Nomor Kartu Kredit), membiarkan aplikasi HP untuk memfilter (*Hide*) data yang tidak perlu.

Jika peretas mem-*Bypass* aplikasi HP dan menginterogasi API menggunakan *Burp Suite*, mereka akan melihat kebocoran seluruh data mentah rahasia tersebut (*Excessive Data Exposure*).

4.2 BOLA (Broken Object Level Authorization)

Kerentanan API paling mematikan (Nomor 1 di OWASP API Top 10) adalah **BOLA**, atau secara tradisional dikenal dengan IDOR (*Insecure Direct Object Reference*).

BOLA terjadi ketika API tidak memverifikasi apakah identitas pengguna yang sedang *login* (Tanda Tangan Sesi) memiliki **HAK/OTORISASI KEPEMILIKAN** terhadap objek data yang mereka minta di URL.

> **Skenario Eksploitasi BOLA:**
> 1. Karyawan "Budi" login secara sah dan mendapat Token.
> 2. Aplikasi memanggil API gajinya: `GET /api/v1/salary/receipt/881` (881 adalah ID Budi).
> 3. Budi memodifikasi permintaan tersebut di peramban, mengganti angka "881" dengan "882" (ID milik Manajer-nya).
> 4. API mengizinkan (*HTTP 200 OK*) dan mengirimkan slip gaji Manajer ke layar Budi.

WAF konvensional buta terhadap BOLA. WAF melihat Token Budi sah, HTTP sah, dan tidak ada muatan SQLi/XSS. Celah murni berada di dalam logika bisnis (*Business Logic*) kode aplikasi.

4.3 KQL: Mendeteksi Anomali Iterasi API

Satu-satunya cara mendeteksi perburuan BOLA secara operasional (jika perbaikan kode belum ada) adalah menangkap upaya **Iterasi ID (ID Enumeration)** yang dilakukan penyerang melalui SIEM.

Saat mencari celah BOLA, penyerang biasanya menggunakan skrip untuk mengubah-ubah ID objek puluhan kali (`/user/101`, `/user/102`, `/user/103`) secara berurutan.

```
/* SIEM KQL: Mendeteksi API Object Enumeration (Indikasi BOLA/IDOR) */ event.dataset:
"api.gateway.access" AND url.path: "/api/v1/receipt/*" /* Hitung jumlah URL spesifik (unik) yang diakses
oleh 1 IP */ | stats count(distinct url.path) as Unique_Endpoints by source.ip, user.name | where
Unique_Endpoints > 25 | timeframe 2m
```

Jika pengguna "Budi" (atau IP-nya) mengakses 25 struk gaji (URL dengan ID berbeda) dalam 2 menit, hal ini tidak normal bagi perilaku aplikasi. Kueri di atas membunyikan alarm peringatan keamanan *API Abuse* bagi SOC untuk mengisolasi akun tersebut.

4.4 API Gateway & Schema Validation

Untuk mengamankan API secara arsitektur, *Security Engineer* menggunakan **API Gateway** (seperti Kong, Apigee, atau AWS API Gateway). Fitur keamanannya yang paling dahsyat adalah **Schema Validation** (Positive Security Model).

Pengembang (*Devs*) menulis cetak biru API mereka menggunakan standar **OpenAPI Specification (OAS / Swagger)**. Fail JSON/YAML OAS ini menguraikan secara kaku bagaimana format *Request* seharusnya: *"Parameter `Umur` harus berupa angka antara 1-100. Parameter `Nama` tidak boleh lebih dari 50 karakter."*

Fail OAS ini diunggah (diimpor) ke *API Gateway*. *Gateway* menggunakannya sebagai tembok besi. Jika seorang peretas mencoba menyuntikkan muatan mematikan (seperti `' OR 1=1`) ke dalam parameter `Umur`, *API Gateway* akan menolaknya (*Drop*) di lapisan terdepan. Serangan dihentikan tanpa perlu WAF menebak apakah itu SQLi atau bukan, karena data *String* telah melanggar format *Integer* (Angka).

4.5 Eksploitasi & Manipulasi JWT

Di era aplikasi Web *Modern/Stateless*, **JSON Web Token (JWT)** menggantikan Session Cookies lawas. JWT dikirim klien melalui tajuk `Authorization: Bearer [TOKEN]`. JWT terdiri dari tiga bagian *Base64* (Header, Payload, Signature).

Ancaman pada JWT adalah kenyataan bahwa *Payload*-nya bisa di-*decode* (dibaca teks jelasnya) oleh siapa saja. Masalah keamanan terjadi saat pengembang tidak melakukan verifikasi **Signature (Tanda Tangan Kriptografi)** dengan benar pada *backend*.

> **Serangan JWT (Alg None):**
> 1. Peretas membaca *Payload* JWT (misal: `"role":"user"`) miliknya.
> 2. Peretas memodifikasinya menjadi `"role":"admin"`.
> 3. Peretas memodifikasi *Header* algoritma kriptografi `"alg":"HS256"` menjadi `"alg":"none"` dan **menghapus Tanda Tangannya**.
> 4. Jika perpustakaan JWT *Backend* salah konfigurasi (rentan CVE), *Backend* akan mengizinkan manipulasi `alg:none` (menganggap token tidak butuh divalidasi), dan peretas mendapat akses Admin instan (*Privilege Escalation*).

4.6 Server-Side Request Forgery (SSRF)

Salah satu vektor *Cloud-Native* yang paling sering dieksploitasi untuk mencuri kunci (*Credentials*) AWS/Azure adalah **Server-Side Request Forgery (SSRF)**. Eksploitasi fatal Capital One tahun 2019 disebabkan oleh SSRF.

Aplikasi web sering kali memiliki fitur yang mengharuskannya mengambil data dari URL luar (misalnya fitur *Webhook* atau *Download Gambar dari URL*). Dalam SSRF, penyerang menyuntikkan (*Submit*) URL internal rahasia ke dalam formulir tersebut.

Karena peladen Web (AWS EC2) Anda memiliki akses penuh ke jaringan di dalam tembok *Virtual Private Cloud* (VPC), peladen Anda yang akan mengambilkan URL rahasia tersebut untuk si penyerang. Sasaran absolut SSRF di *Cloud* adalah **Metadata Service Endpoint AWS (IP Magis:** `169.254.169.254`**)**. Jika penyerang menyuruh aplikasi membaca URL tersebut, aplikasi akan menampilkan Kunci Rahasia *Cloud* (*AWS IAM Temporary Credentials*) peladen Anda langsung ke layar peretas.

4.7 KQL: Deteksi SSRF pada Log WAF

Mendeteksi SSRF membutuhkan konfigurasi WAF untuk menangkap upaya resolusi alamat IP (*Private IP Spaces*) dan IP Metadata Cloud ke dalam *Query Parameters* (Parameter URL) yang tidak seharusnya.

```
/* SIEM KQL: Deteksi Payload SSRF ke Cloud Metadata & Private IP */ event.dataset: "waf.access" AND
url.query: ( /* Magic IP AWS/GCP Metadata */ "*169.254.169.254*" OR /* Localhost / Loopback bypasses */
"*127.0.0.1*" OR "*localhost*" OR "*0.0.0.0*" OR /* AWS IAM path request */ "*latest/meta-
data/iam/security-credentials*" )
```

Penyerang mungkin mencoba *encoding* URL atau trik heksadesimal (misal `0xA9FEA9FE` untuk 169.254.169.254) untuk memintas *regex* teks biasa. *Engine WAF* Lapis 7 (Advanced WAF) secara otomatis membongkar (*decode*) ini sebelum meneruskan pola ke sistem analitik Anda.

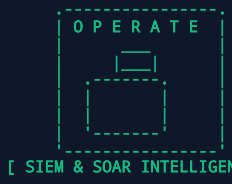
4.8 Keamanan GraphQL (Introspection Abuse)

GraphQL adalah teknologi kueri modern (pengganti REST) yang memungkinkan aplikasi klien (*Frontend*) mengambil berbagai potongan data dari banyak tabel *Database* hanya dengan satu permintaan (*Single Endpoint:* `/graphql`).

Risiko operasional terbesar GraphQL di masa produksi (*Production*) adalah dibiarkannya fitur **Introspection (Introspeksi)** tetap menyala (*Enable*). Introspeksi adalah fitur *Debugging* pengembang.

Jika fitur ini menyala di internet publik, peretas dapat mengirim kueri khusus `__schema` ke *server*. *Server GraphQL* akan membalas dengan memuntahkan **Seluruh Cetak Biru (Blueprint)** struktur aplikasi web Anda: Seluruh nama kueri tersembunyi, skema *Database*, dan struktur mutasi admin.

Dalam 2 detik, peretas memiliki peta harta karun eksak untuk serangan injeksi (*Precision Strikes*). Tindakan SOC yang krusial: Bekerja dengan tim *DevSecOps* untuk mematikan mode Introspeksi (*Disable GraphQL Introspection*) di tingkat *Gateway Server*.



BAGIAN V

SOC Operations: Analitik, SOAR,
& Threat Intelligence

BAB 5: SOC & SOAR OPERATIONS

5.1 Ingesting Web Logs (NGINX/CDN) ke SIEM

Keberhasilan pertahanan perimeter dan perburuan ancaman (*Threat Hunting*) di SIEM bergantung pada kualitas *Ingestion Pipeline* (masukan log). Jika Anda menggunakan peladen *Origin* berbasis *Nginx*, *Apache*, atau IIS tanpa modifikasi format log, Anda akan menyusahkan tim *Security* Anda sendiri.

Format log akses *Nginx* tradisional (*Combined Log Format*) adalah teks mentah (*Raw String*). Mengekstrak parameter darinya menggunakan *Regex Grok Parser* di sisi logstash (SIEM) membutuhkan sumber daya komputasi tinggi (CPU).

Praktik Terbaik Industri: Tim DevSecOps HARUS memodifikasi berkas `nginx.conf` pada *server* produksi agar mengirimkan log secara langsung ke SIEM (atau FileBeat/Splunk Universal Forwarder) dalam format **JSON (JavaScript Object Notation) Tersusun (Structured)**.

Dengan JSON Log, *Field* seperti `http.request.method` atau `url.path` sudah dipecah rapi, membuat pencarian analitik serangan 100x lebih cepat (*Query Optimization*).

5.2 Normalisasi ECS (Elastic Common Schema)

Perusahaan modern biasanya tidak memiliki 1 jenis *Web Server*. Mereka memiliki WAF F5 di *On-Premise*, Cloudflare di *Edge*, dan *Nginx* di *Kubernetes* (K8s). Masing-masing sistem ini memberi nama *Field* yang berbeda untuk hal yang sama (Ada yang menyebut alamat IP dengan `clientIP`, `src_ip`, atau `remote_addr`).

Jika analis SOC harus menulis aturan dengan nama lapangan (*field*) yang berbeda-beda untuk setiap log, hal ini tidak *scalable* (mudah rusak). Solusinya adalah **Normalisasi Skema** (seperti penerapan *Elastic Common Schema / ECS*).

> **Standardisasi Medan Web (ECS Web Fields):**
> - Seluruh IP Penyerang dari vendor manapun dinormalkan menjadi: `source.ip`
> - *User Agent* menjadi: `user_agent.original`
> - Tujuan Target (*Host*) menjadi: `url.domain`
> - Kode status (misal 404/200) menjadi: `http.response.status_code`

Dengan ECS, satu baris aturan KQL `source.ip: 10.0.0.1 AND http.response.status_code: 404` akan otomatis memeriksa log dari Nginx, IIS, dan Cloudflare secara bersamaan.

5.3 Merancang Dashboard Visibilitas Perimeter

Dashboard keamanan bukan untuk dipandangi seharian, melainkan untuk memberikan kesadaran situasional kilat (*Single Pane of Glass*) saat krisis DDoS/Serangan terjadi. *Dashboard* Pertahanan Tepi (*Edge Defense*) yang baik (Kibana/Splunk) memuat visualisasi metrik agregat berikut:

- **WAF Block Action Rate (Grafik Garis):** Tren aktivitas WAF memblokir permintaan per jam. Lonjakan tinggi menandakan *Scanner* massal sedang bekerja.
- **Top 10 Blocked Source IPs (Tabel Data):** Identifikasi *Threat Actors* paling gigih pada hari ini.
- **Geo-Map Heatmap (Peta Distribusi):** Pemetaan IP penyerang dari seluruh dunia. Jika layanan Anda lokal (Indonesia), trafik masif dari luar negeri memudahkan Anda memutuskan penyalakan blokir *Geo-Fencing* (Blokir Negara).
- **HTTP Error Ratios (Grafik Donat / Pie Chart):** Proporsi respons 200 vs 404/500/403. Persentase HTTP 404/403 yang mendominasi menunjukkan *Brute Force / Reconnaissance* parah (Bab 2.3).

5.4 Otomatisasi SOAR: WAF Blocklist Update

Di perimeter, peringatan berjumlah ribuan. Tim *Security* L1 tidak mungkin masuk ke *Dashboard* WAF dan mengetik IP satu per satu untuk diblokir. Semuanya ditenagai oleh *Security Orchestration, Automation, and Response (SOAR)* (Bab 2.9).

```
# Pseudo-Python (Logika SOAR Integration Cloudflare/AWS WAF API) import requests def
block_ip_on_edge(malicious_ip): api_url =
"https://api.cloudflare.com/client/v4/zones/.../firewall/access_rules/rules" headers = {"Authorization":
"Bearer WAF_API_TOKEN"} payload = { "mode": "block", # Opsi lain: "challenge" / "js_challenge"
"configuration": { "target": "ip", "value": malicious_ip }, "notes": "Auto-Blocked by SIEM Alert (SQLi
Spike)" } requests.post(api_url, json=payload, headers=headers)
```

Melalui API ini, SOAR (seperti alat Tines, Shuffle, atau XSOAR) menyatukan deteksi *Splunk* dan menjatuhkan palu blokade di Cloudflare dalam waktu 2 detik paska-deteksi, membuktikan perlindungan L7 Otonom (*Autonomous Defense*).

5.5 External Dynamic Lists (EDL) & Threat Intel

Kebijakan pemblokiran *Web* tercanggih bersifat proaktif, mendahului (*Pre-emptive*) tindakan *malware* menyentuh server Anda. Arsitek *Security* memanfaatkan sinkronisasi daftar hitam global bernama **External Dynamic Lists (EDL)** atau Feed *Threat Intelligence (TI)*.

Organisasi keamanan global (seperti *Spamhaus*, *AbuseIPDB*, *GreyNoise*) mengumpulkan dan mempublikasikan daftar alamat IP internet publik yang **saat ini (hari ini)** aktif melakukan operasi *Botnet*, *Scanning*, atau menjadi relai jaringan *Tor (Exit Nodes)*.

Edge WAF/Firewall dikonfigurasi untuk secara otomatis mengunduh daftar teks berukuran *Megabytes* ini (mengandung puluhan ribu IP) setiap 5 menit (*Pull API*). Aturan *Firewall Gateway* berbunyi: *"Jika IP masuk berasal dari daftar EDL Tor_Nodes atau Botnet_Active_List, jatuhkan paket (DROP)."*

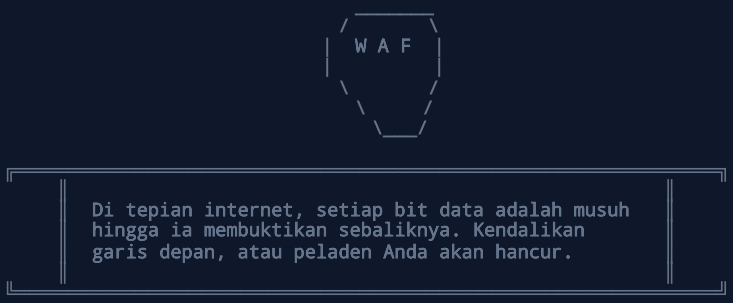
Ini adalah pertahanan *Zero-Day* Lapis 3. Penyerang dihentikan secara permanen bahkan sebelum mereka dapat mengirimkan tahap *HTTP GET* menuju aset kritis Anda.

GLOSARIUM WEB SECURITY (A-M)

TERMINOLOGI	DEFINISI & KONTEKS SERANGAN
API Gateway	Infrastruktur sentral yang mengontrol perutean, manajemen laju (*Rate Limit*), dan autentikasi (Validasi Skema/OAS) sebelum paket mencapai *Microservices backend*.
BOLA (IDOR)	*Broken Object Level Authorization*. Kerentanan L7 saat penyerang mengubah parameter ID Objek di URL (misal `/user/2` ke `/user/3`) untuk mencuri data lintas-kepemilikan.
CDN / Edge	*Content Delivery Network* (misal: Cloudflare, Akamai). Kumpulan *server* global terdistribusi. Menyelamatkan infrastruktur dari DDoS volumetrik L3/L4 dan menyediakan lokasi pengerjaan WAF.
Credential Stuffing	Serangan terotomatisasi (Botnet) menggunakan kumpulan nama pengguna/kata sandi bocoran (*Dumped Hashes*) untuk menguji ribuan situs guna membajak akun (*Account Takeover*).
JA3 / JA3S	Sistem pencetakan sidik jari kriptografik (*TLS Fingerprinting*) yang mengekstrak nilai matematika dari *Client Hello* untuk mengidentifikasi *Malware/Bot* secara pasif (tanpa bergantung pada IP).

GLOSARIUM WEB SECURITY (N-Z) & PENUTUP

TERMINOLOGI	DEFINISI & KONTEKS SERANGAN
Positive Security	(*Whitelisting*). Model kebijakan pertahanan di mana aplikasi memblokir SEMUA input secara absolut, kecuali data yang berformat eksak sesuai spesifikasi ketat sistem (Skema API).
Rate Limiting	Fungsi WAF/Gateway untuk membatasi frekuensi (*Request Rate*) lalu lintas dari satu IP/Entitas dalam jeda waktu tertentu untuk mematikan eksploitasi *Brute Force / DDoS L7*.
SSRF	*Server-Side Request Forgery*. Penyerang menyuntikkan URL agar peladen *Backend* membuat koneksi (Bypass). Sering digunakan untuk mencuri Kredensial Metadata *Cloud* rahasia AWS/Azure (169.254.169.254).
User-Agent	Tajuk (*Header*) HTTP identifikasi peramban/klien. Sangat mudah dipalsukan (*Spoofed*) namun rentan dimanipulasi dengan buruk oleh skrip (*Python-urllib*) yang memudahkan identifikasi.
WAF	*Web Application Firewall*. Filter Lapis 7 yang membaca parameter HTTP, Headers, dan Payload URI untuk menahan injeksi SQLi, XSS, RCE, dan Traversal sebelum diproses *Backend*.



The Digital Shield

© 2026 All rights reserved.
SOC Training & Reference Material.